MINI MACHINE LEARNING PROJECT

Handwritten Digit Recognition

A CNN-based desktop application

Project type	Machine-learning desktop application
Model	Convolutional Neural Network (CNN)
Datasets	MNIST and EMNIST Digits
User input	One hand-drawn digit at a time
Application	Python and Tkinter
Distribution	Packaged Windows executable

Prepared by: *[Mohit]*

Department / Institution: *[IIT Hyderabad]*

Date: *[Oct. 2026]*

Abstract

This project implements a desktop application for handwritten digit recognition using a Convolutional Neural Network (CNN). The training script first trains the network on the MNIST dataset and then continues training the same model on the digit-only subset of EMNIST. The resulting model is saved as `digit_model_improved.keras` and loaded by a Tkinter-based drawing application. A user draws a single digit on a canvas and requests a prediction among ten classes, from 0 to 9.

The project brings together dataset preparation, CNN training, model evaluation, graphical user interaction, and packaging for Windows. Initial testing with user-drawn digits showed that practical recognition was less reliable than desired. This highlights an important distinction: good performance on benchmark datasets does not guarantee equally good performance on drawings produced through a different input interface. The report describes the implemented system, its workflow, observed limitation, and the next steps for improving and measuring real-world performance.

Keywords: Machine learning, CNN, MNIST, EMNIST, digit classification, TensorFlow, Tkinter.

Contents

Abstract i

1 Introduction 1
- 1.1 Project Background . . . 1
- 1.2 Objectives . . . 1
- 1.3 Scope . . . 1

2 Datasets and Preprocessing 2
- 2.1 MNIST . . . 2
- 2.2 EMNIST Digits . . . 2
- 2.3 Image Normalization . . . 2

3 CNN Model Architecture 4
- 3.1 Architecture Overview . . . 4
- 3.2 Layer Descriptions . . . 5
- 3.3 Compilation . . . 5

4 Training and Evaluation 6
- 4.1 Two-Stage Training Strategy . . . 6
  - 4.1.1 Stage 1: Training on MNIST . . . 6
  - 4.1.2 Stage 2: Additional Training on EMNIST . . . 6
  - 4.1.3 Continuing Training . . . 6
- 4.2 Evaluation Metrics . . . 6
- 4.3 Important Evaluation Limitation . . . 7

5 Desktop Application and Packaging 8
- 5.1 Tkinter Interface . . . 8
- 5.2 Windows Packaging . . . 8

6 Testing, Limitations, and Future Work 9
- 6.1 Observed Behaviour . . . 9
- 6.2 Possible Factors Affecting Real-World Predictions . . . 9
- 6.3 Planned Improvements . . . 9

7 Conclusion 11

References 12

Chapter 1

Introduction

1.1 Project Background

Handwritten digit recognition is a common introductory application of image classification. The task is to assign an input image containing one handwritten digit to one of ten classes, labelled 0 through 9. Although the task appears simple to a human, handwriting varies in shape, slant, scale, stroke thickness, and position.

This project implements a complete small-scale machine-learning application rather than only training a model in a notebook. A CNN is trained using public digit datasets, saved to a model file, and connected to a graphical interface where the user can draw a digit and obtain a prediction.

1.2 Objectives

The main objectives are:

- Build a CNN for ten-class handwritten digit classification.
- Train the model using MNIST, followed by additional training on EMNIST Digits.
- Evaluate the saved model on the test splits of both datasets.
- Connect the trained model to an interactive Tkinter drawing interface.
- Package the application for distribution on Windows.
- Identify the gap between dataset evaluation and performance on user-drawn examples.

1.3 Scope

The current application predicts **one digit per input**. It is not designed to read multi-digit numbers, words, or complete handwritten documents. The project focuses on the machine-learning pipeline and its integration into a desktop application.

Chapter 2

Datasets and Preprocessing

2.1 MNIST

MNIST is a standard dataset for handwritten digit classification. It contains 70,000 grayscale images, each with a resolution of 28×28 pixels. The conventional split contains 60,000 training images and 10,000 test images. Each image has an integer label from 0 to 9.

In the supplied code, MNIST is loaded using:

```
(x_train, y_train), (x_test, y_test) = \
    tf.keras.datasets.mnist.load_data()
```

2.2 EMNIST Digits

The second dataset is the digit-only EMNIST split, loaded using TensorFlow Datasets with the identifier `emnist/digits`. It provides additional examples of handwritten digits from a dataset source different from MNIST.

The code transposes each EMNIST image before adding it to the dataset arrays. This is an orientation correction used by the implementation because the image layout returned by EMNIST can differ from the orientation expected by the model.

Table 2.1: *Datasets used in the training workflow*

Dataset	Purpose	Representation
MNIST	Initial model training and test evaluation	28×28 grayscale images
EMNIST Digits	Additional training and separate test evaluation	Transposed 28×28 grayscale images

2.3 Image Normalization

Both datasets are converted to `float32` and normalized by dividing each pixel value by 255. This scales pixel intensities from the range 0–255 to the range 0–1. A channel dimension is then added to each image.

$$x_{\text{normalized}} = \frac{x_{\text{pixel}}}{255} \tag{2.1}$$

The resulting input shape for a single image is $28 \times 28 \times 1$, where the final dimension represents one grayscale channel.

Chapter 3

CNN Model Architecture

3.1 Architecture Overview

The model is defined as a sequential CNN in the `create_model()` function. Convolutional layers learn spatial patterns in images; max-pooling layers reduce the spatial dimensions; and dense layers use the extracted features to classify the image.

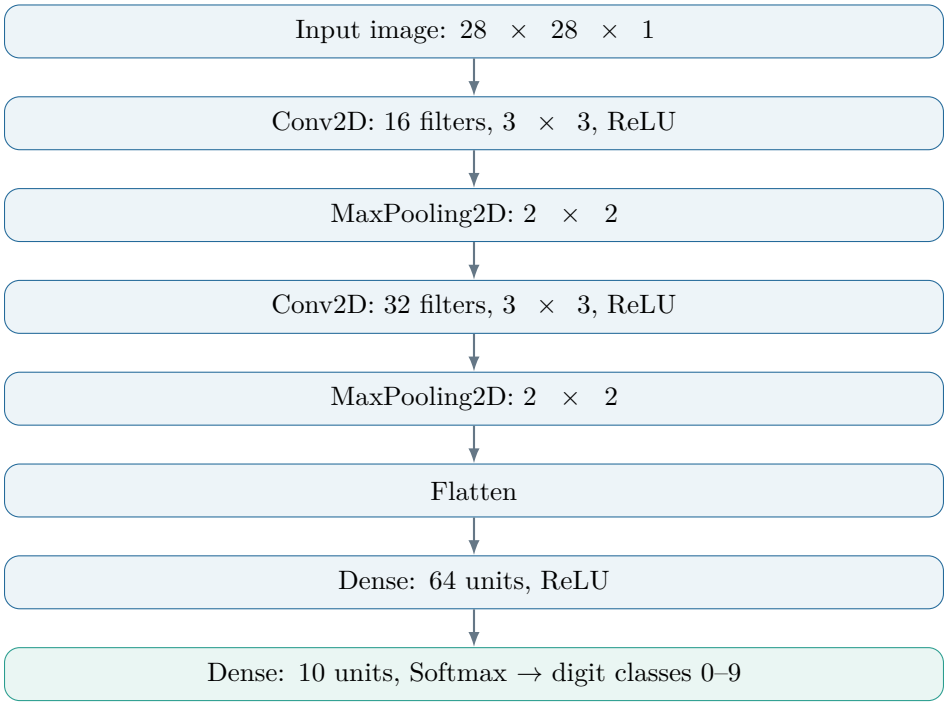


Figure 3.1: *Layer sequence of the CNN implemented in the training script.*

3.2 Layer Descriptions

Table 3.1: *CNN architecture and role of each layer*

Layer	Configuration	Role
Input	$28 \times 28 \times 1$	Receives a normalized grayscale image.
Convolution 1	16 filters, 3×3, ReLU	Learns basic features such as edges and stroke patterns.
Max pooling 1	2×2	Reduces spatial dimensions.
Convolution 2	32 filters, 3×3, ReLU	Learns more complex combinations of visual features.
Max pooling 2	2×2	Further reduces spatial dimensions.
Flatten	Vector conversion	Converts feature maps into a one-dimensional feature vector.
Dense	64 units, ReLU	Combines learned features for classification.
Output	10 units, softmax	Produces class probabilities for digits 0–9.

3.3 Compilation

The network is compiled with the Adam optimizer, sparse categorical cross-entropy loss, and accuracy as a tracked metric. Sparse categorical cross-entropy allows the class labels to remain integers rather than being converted to one-hot vectors.

Chapter 4

Training and Evaluation

4.1 Two-Stage Training Strategy

The script supports training from scratch and continuing training from a previously saved improved model.

4.1.1 Stage 1: Training on MNIST

When option 1 is selected, a new CNN is created and trained on MNIST. The user chooses the number of epochs. The code uses a batch size of 32 and reserves 10% of the training data for validation. After this stage, the model is saved to `digit_model_improved.keras`.

4.1.2 Stage 2: Additional Training on EMNIST

The same model is then trained further on the EMNIST Digits training split. The same user-specified epoch count, batch size of 32, and 10% validation split are used. The updated model is saved to the improved-model path, replacing the previous saved version.

4.1.3 Continuing Training

Option 2 loads the existing improved model and continues training it on EMNIST. It does not repeat the MNIST training stage. Option 3 loads the saved model and evaluates it on both test sets.

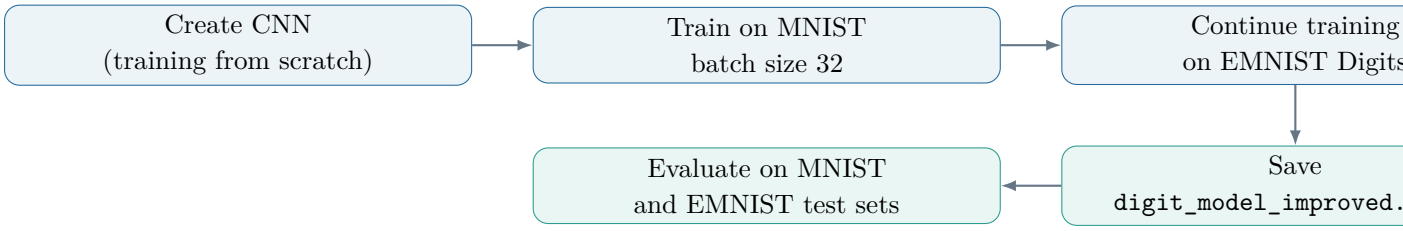


Figure 4.1: *Training and evaluation sequence when starting from a new model.*

4.2 Evaluation Metrics

The script reports test loss and accuracy for both datasets. Accuracy is the proportion of correctly classified images in the relevant test set:

$$Accuracy = \frac{Numberofcorrectpredictions}{Totalnumberoftestexamples} \times 100\%. \tag{4.1}$$

4.3 Important Evaluation Limitation

The MNIST and EMNIST test results measure performance on those datasets. They do not directly measure accuracy on digits drawn in the application's canvas. A separate test set of user-drawn digits is needed to measure real-world performance.

Chapter 5

Desktop Application and Packaging

5.1 Tkinter Interface

The application in `draw_digit.py` uses Tkinter to provide a drawing canvas and prediction interaction. It loads the saved `digit_model_improved.keras` model and uses it to classify a drawing. The expected task is one digit per prediction, with an output class between 0 and 9.

The input preparation performed by the application is important. The drawing must be converted into a representation compatible with the training images, including image size, grayscale format, pixel scaling, foreground/background polarity, and positioning. A mismatch at this stage can reduce prediction quality even if the model performs well on the benchmark test data.

5.2 Windows Packaging

The application was packaged using PyInstaller in an `onedir` distribution. In this format, the executable is distributed with its supporting runtime files in a folder. The folder was compressed into a ZIP archive and published as a GitHub release.

A user can download and extract the ZIP, then run the executable. Python and the project dependencies do not need to be installed separately by the end user, provided the complete packaged folder is retained.

Chapter 6

Testing, Limitations, and Future Work

6.1 Observed Behaviour

The application launches and supports the intended draw-and-predict workflow. However, testing with real hand-drawn digits showed that recognition was less reliable than desired. This is the main practical limitation identified so far.

6.2 Possible Factors Affecting Real-World Predictions

- **Image polarity:** the GUI may produce dark strokes on a light background, while the model expects the opposite convention.
- **Scale and centering:** a digit that fills too much or too little of the image, or is off-centre, may differ from the training examples.
- **Resizing:** resizing the entire canvas directly to 28×28 can distort the digit or preserve excessive empty margins.
- **Stroke thickness:** mouse-drawn lines can be thicker or thinner than the strokes in the datasets.
- **Dataset shift:** the handwriting styles and image distributions in MNIST, EMNIST, and user drawings are not identical.
- **Sequential fine-tuning:** training on EMNIST after MNIST can change performance on MNIST, so both test sets should continue to be monitored.

6.3 Planned Improvements

1. Inspect and standardize the GUI preprocessing: grayscale conversion, polarity, aspect-preserving resizing, centering, and normalization.
2. Create a small labelled test set of digits drawn through the actual application.
3. Compare the MNIST-only model and the MNIST-then-EMNIST model on the same test sets.
4. Use a confusion matrix and example predictions to identify which digits are commonly confused.
5. Record the actual test metrics and real-drawing accuracy in the report.

6. Retain the current interface and replace the model only after the updated version has been tested.

Chapter 7

Conclusion

This project demonstrates an end-to-end machine-learning application for single handwritten-digit recognition. The implementation uses a CNN trained first on MNIST and then further trained on EMNIST Digits. The saved Keras model is integrated with a Tkinter drawing interface and distributed as a Windows executable using PyInstaller.

The application provides a working foundation, while real-drawing tests have revealed a gap between dataset performance and practical use. The next development phase should focus on matching GUI preprocessing to the training data and measuring performance on a dedicated set of user-created digits. These steps will provide a clearer basis for judging whether model changes genuinely improve the application.

References

1. LeCun, Y., Cortes, C., and Burges, C. J. C. *The MNIST Database of Handwritten Digits*. Available at: http://yann.lecun.com/exdb/mnist/

2. TensorFlow. *TensorFlow and Keras documentation*. Available at: https://www.tensorflow.org/

3. TensorFlow. *TensorFlow Datasets documentation*. Available at: https://www.tensorflow.org/datasets

4. Python Software Foundation. *Tkinter documentation*. Available at: https://docs.python.org/3/library/tkinter.html

5. PyInstaller. *PyInstaller documentation*. Available at: https://pyinstaller.org/

— End of Report —